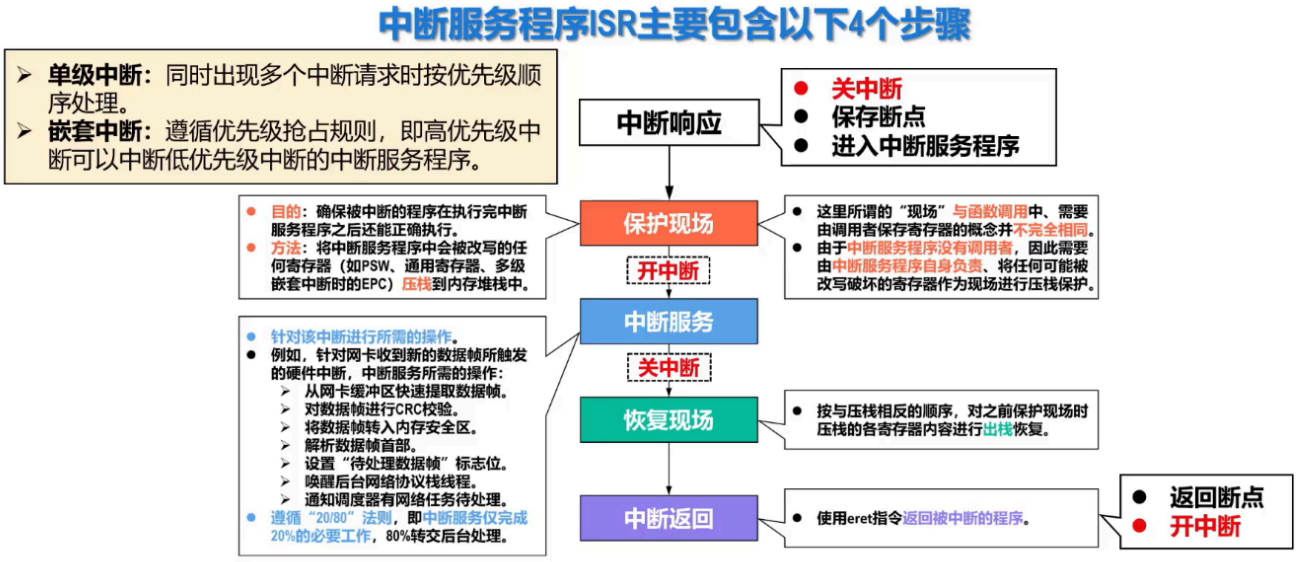


对于多级嵌套中断，保护现场后应开中断（以确保中断服务程序还可以被中断），恢复现场前应关中断（以确保恢复现场的完整性）。

中断响应→保护现场→中断服务→恢复现场→中断返回五个环节（前两者和后两者之间插入开/关中断），其中**中断响应由硬件自动完成**（即中断隐指令），**后面四步全部由软件（ISR代码）完成**

一、中断响应：硬件阶段的三件事

这是ISR的入口，对应中断隐指令，**完全由硬件在一个响应周期内原子完成**：

1. **关中断**：IE←0，防止保存断点的过程被新中断打断
2. **保存断点**：PC→EPC（或压栈），记录返回位置
3. **进入中断服务程序**：通过中断向量表或MUX选择逻辑，将ISR入口地址装入PC

这三步在数据通路图上对应"操作控制器发出的四条控制信号"，在微程序控制器图上对应"中断响应微程序"的逐条微指令——硬件行为本质上是微程序化的

二、保护现场：软件接力的第一步

目的与方法

确保被中断程序在ISR执行完后**还能无缝继续。**方法是将ISR中**会被改写的任一寄存器**（PSW、通用寄存器、多级嵌套时的EPC）压栈到内存堆栈

关键辨析："现场"≠ 函数调用的"调用者保存寄存器"

图中特别强调的概念辨析非常重要：

- **函数调用场景**：调用者知道被调用函数会破坏哪些寄存器（ABI约定），因此由调用者提前保存自己关心的寄存器
- **中断场景**：ISR**没有"调用者"**——被打断的程序完全不知道自己会被在哪里打断、被什么程序打断，无法提前保存任何寄存器

因此**ISR必须自己负责**，把任何可能被自己破坏的寄存器全部压栈保存——这是中断现场保护与普通函数调用最本质的区别

三、开中断：嵌套中断的分水岭

保护现场完成后立即开中断（IE←1），这是**支持中断嵌套的关键设计**：
- **保护现场前必须关中断**：压栈过程若被打断，栈指针和已压栈内容会出现不一致，现场保护不完整
- **保护现场后立即开中断**：让更高优先级中断能够抢占进来，缩短高优先级事件的响应延迟

左上角说明了两种中断处理策略：
- **单级中断**：多个中断同时到达时按优先级**串行排队处理**，ISR执行期间不允许被打断
- **嵌套中断**：遵循**优先级抢占规则**——高优先级中断可以打断正在执行的低优先级ISR，形成"ISR套ISR"的嵌套链

四、中断服务：真正的业务逻辑

这一步执行针对具体中断的实际处理，以**网卡收到数据帧**为例的七项操作：

- 从网卡缓冲区快速提取数据帧
- 对数据帧进行CRC校验
- 将数据帧转入内存安全区
- 解析数据帧首部
- 设置"待处理数据帧"标志位
- 唤醒后台网络协议栈线程
- 通知调度器有网络任务待处理

20、80法则：中断服务的设计哲学

图中特别标注遵循"**20/80法则**"——ISR只完成**20%的必要工作**（紧急、时效性强的部分），剩余**80%转交后台处理**（唤醒协议栈线程去慢慢处理）

这个设计背后的逻辑是：**ISR执行时间越长，其他中断（包括更高优先级的）被阻塞的时间就越长**，系统实时性越差。因此ISR要"快进快出"，只做必须在中断上下文完成的事，把耗时工作（如协议解析、数据处理）丢给后台线程

五、关中断 + 恢复现场

中断服务结束后立即**关中断**（IE←0），原因与保护现场时完全对称：
- 恢复现场是**压栈的逆过程**：按与压栈相反的顺序（LIFO），将之前压栈的各寄存器内容依次出栈
- 若恢复过程中被新中断打断，栈中尚未恢复的寄存器内容会被新ISR破坏，导致原程序恢复后寄存器值错乱

恢复现场的完整性依赖关中断保护——这是为什么图中强调"恢复现场前应关中断"的根本原因

六、中断返回：硬件收尾

执行**eret指令**返回被中断的程序，这条指令触发硬件动作：
- **返回断点**：EPC→PC，恢复被打断程序的下一条指令地址
- **开中断**：IE←1，允许后续中断响应

注意**开中断放在中断返回这一步**，而不是恢复现场后手动STI——因为eret需要原子地完成"恢复PC + 开中断"，若开中断与恢复PC之间被插入新中断，断点地址会错乱

七、完整时序与开/关中断的对称性

```
中断响应（硬件）：关中断 → 保存断点 → 跳ISR入口
    ↓
保护现场（软件）：压栈保存会被破坏的寄存器
    ↓
开中断            ← 仅嵌套中断需要，放行高优先级
    ↓
中断服务（软件）：20%紧急工作，80%转交后台
    ↓
关中断            ← 防止恢复现场被打断
    ↓
恢复现场（软件）：按LIFO顺序出栈恢复
    ↓
中断返回（eret）：EPC→PC + 开中断，回到断点
```

开与关的对称性是ISR设计的精髓：
- **保护现场前关、恢复现场前关**——两次"关"都是为了保护**栈操作的原子性**
- **保护现场后开、eret中开**——两次"开"都是为了**不被不必要地阻塞**

八、一句话总结

ISR是"硬件铺路（中断响应）+ 软件走路（保护、服务、恢复）+ 硬件收尾（eret）"的三段式结构，其中**开/关中断的四个时机**（响应后关、保护后开、服务后关、返回时开）构成一组严丝合缝的对称设计，既保证现场操作原子性，又最大限度不阻塞更高优先级中断——这就是408反复考查的"中断嵌套与屏蔽字"在工程层面的具体落地